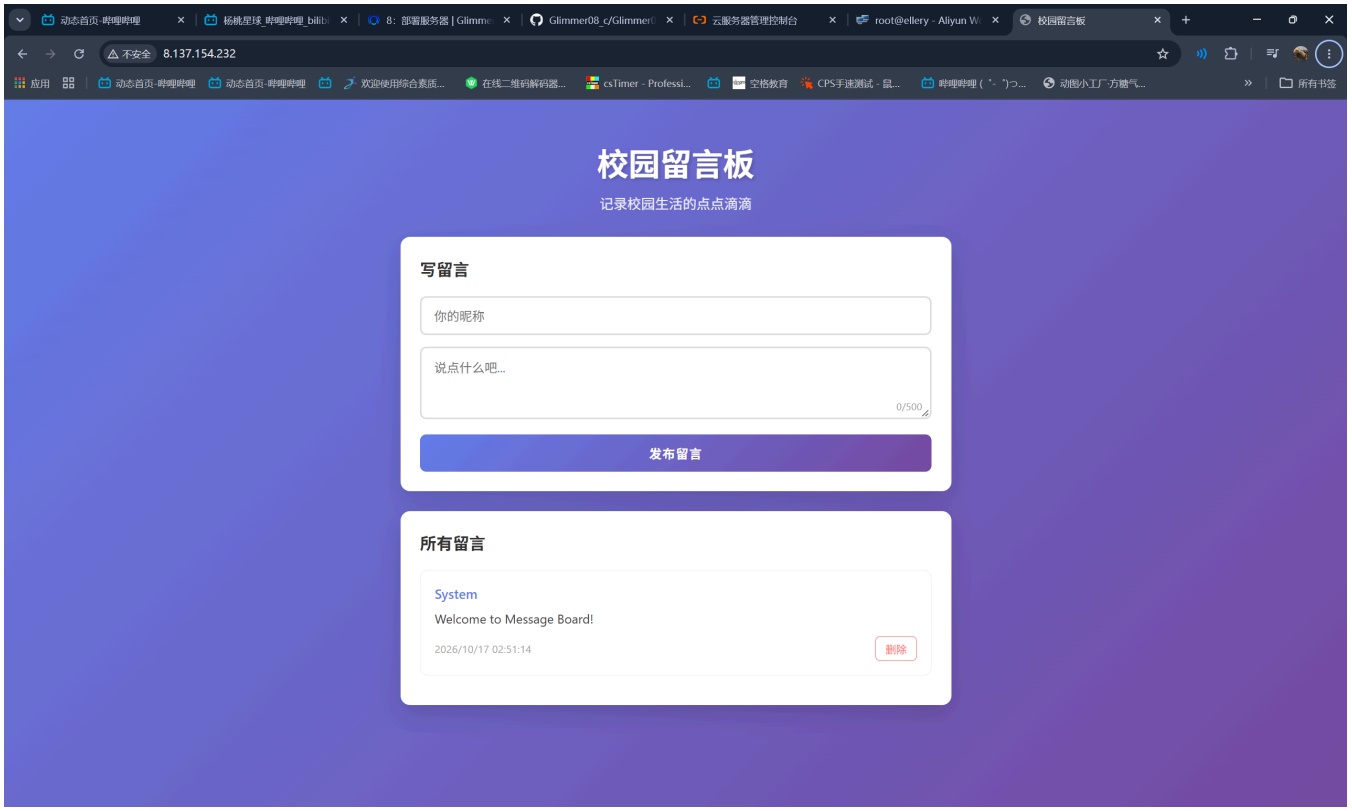
GitHub仓库链接

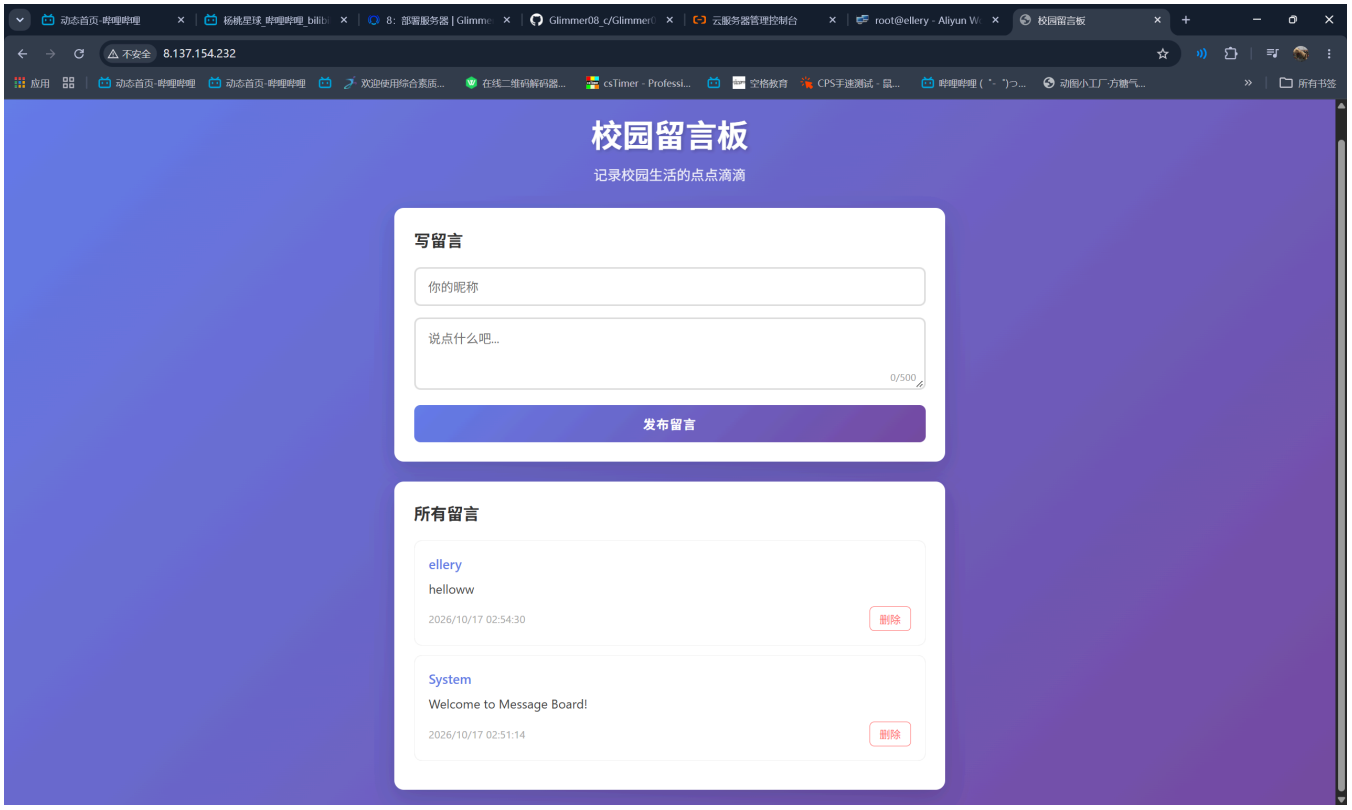
https://github.com/ellery-jpg/Glimmer08_c

相关截图

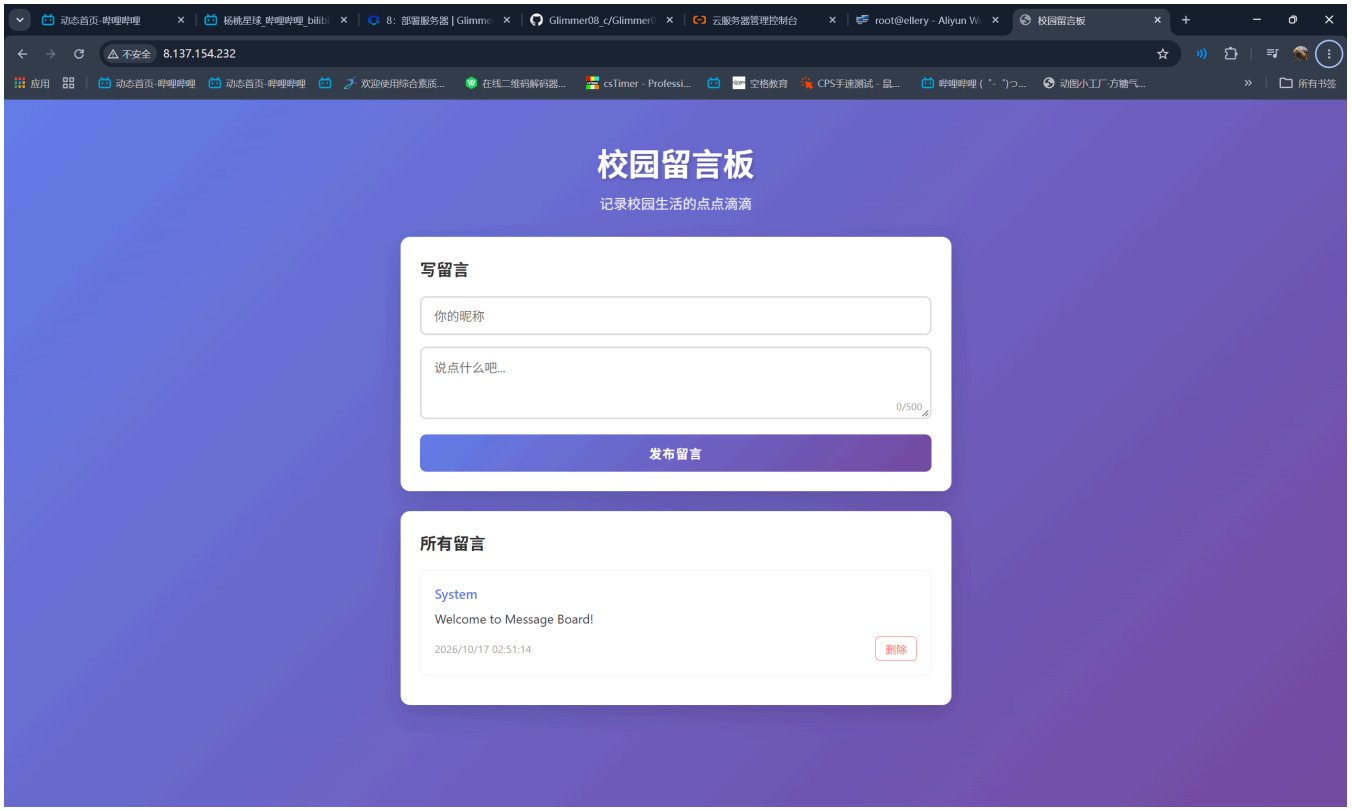
成功进入网页



添加一个留言


校园留言板
记录校园生活的点点滴滴
写留言
你的昵称
说点什么吧...
0/500
发布留言
所有留言
ellery
helloww
2026/10/17 02:54:30
删除
System
Welcome to Message Board!
2026/10/17 02:51:14
删除

删除留言

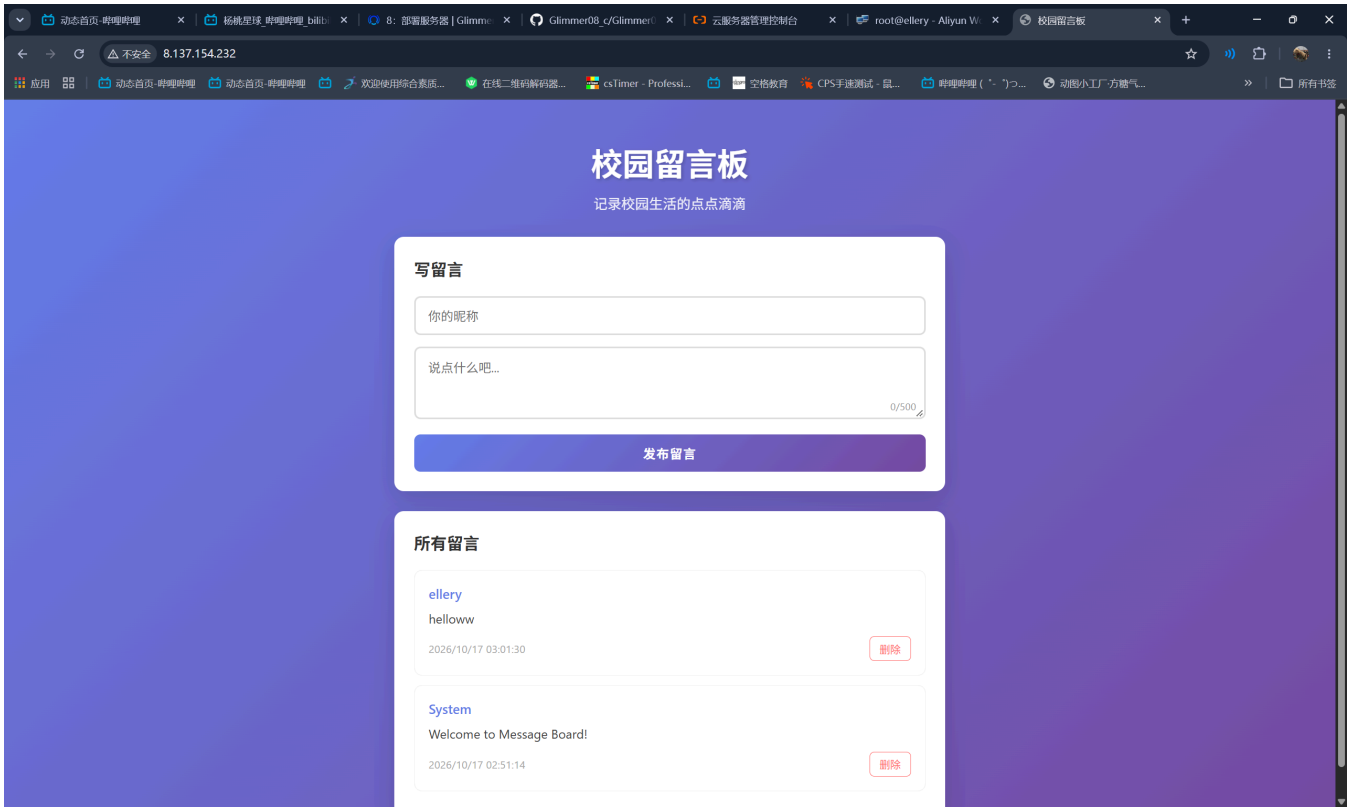

校园留言板
记录校园生活的点点滴滴
写留言
你的昵称
说点什么吧...
0/500
发布留言
所有留言
System
Welcome to Message Board!
2026/10/17 02:51:14
删除

数据持久性的测试（我又添加了一个相同的留言）

停止、销毁容器

```
root@iZ2vcglekxrvvx76802u6mZ:/opt/Glimmer08_c/Glimmer08# docker compose down
[+] Running 4/4
 ✓ Container glimmer08-nginx-1    Removed
 ✓ Container glimmer08-backend-1  Removed
 ✓ Container glimmer08-mysql-1    Removed
 ✓ Network glimmer08_default      Removed
root@iZ2vcglekxrvvx76802u6mZ:/opt/Glimmer08_c/Glimmer08# docker compose up -d
[+] Running 4/4
 ✓ Network glimmer08_default      Created
 ✓ Container glimmer08-mysql-1    Healthy
 ✓ Container glimmer08-backend-1  Started
 ✓ Container glimmer08-nginx-1    Started
root@iZ2vcglekxrvvx76802u6mZ:/opt/Glimmer08_c/Glimmer08# 输入自然语言或命令，AI将智能响应，试试打个招呼吧
```

重新启动发现数据还在



配置文件

- **Dockerfile**

  主要是复制jar包到工作目录并启动

```
#来自jdk17
FROM eclipse-temurin:17
#工作目录：/app
WORKDIR /app
#把jar包复制
COPY target/message-board-1.0.0.jar message-board.jar
#启动jar包
ENTRYPOINT ["java","-jar","message-board.jar"]
```

- **docker-compose.yml**

  设定三个容器的启动顺序，和一些设置

  mysql的环境变量、数据挂载（数据库和建表脚本）和健康检查

  后端容器的服务连接，利用compose的共享虚拟网桥，直接使用容器名可以作为服务名

  nginx的端口、数据挂载（网页文件和配置文件）

```
services:
 mysql:
  image: mysql:8.4
  environment:
   MYSQL_ROOT_PASSWORD: "123456"
   MYSQL_DATABASE: message_board
  volumes:
   - ./mysql_data:/var/lib/mysql     #把数据挂载到数据卷
   - ./sql/init.sql:/docker-entrypoint-initdb.d/init.sql     #把初始化脚本挂载到docker初始化脚本目录
  healthcheck:
   test: ["CMD","mysqladmin","ping","-h","localhost","-uroot","-p123456"]
   interval: 10s     #每10秒检查一次
   timeout: 5s       #检查间最多等待5秒
   retries: 10       #连续10次后仍失败，标记为不健康


 backend:
  build: ./backend
  environment:
   SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/message_board
   SPRING_DATASOURCE_USERNAME: root
   SPRING_DATASOURCE_PASSWORD: "123456"
  depends_on:
   mysql:
    condition: service_healthy

 nginx:
  image: nginx:alpine
  ports:
   - "80:80"
  volumes:
   - ./frontend:/usr/share/nginx/html    #把项目网页文件提供给 Nginx,这样看到的不会是nginx的欢迎
   - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf   #让容器的nginx使用我的nginx配置
  depends_on:
   - backend
```

- **nginx.conf**

  原来已经有了一个静态网页location块

  这里补上一个location块，处理网页接口请求，反向代理转发请求到后端，，8080端口

```
server {
```

```
    listen       80;
    listen  [::]:80;
    server_name  localhost;
    location / {
        root   /usr/share/nginx/html;
        index  index.html index.htm;
    }
    #仅允许api开头的请求，也就是接口请求
    location /api/ {
        proxy_pass http://backend:8080; #反向代理，把请求转发到服务器
    }
    error_page   500 502 503 504  /50x.html;
    location = /50x.html {
        root   /usr/share/nginx/html;
    }
}
```